Graph-Native Coding Harness

Does giving a coding agent a code graph actually help — and *how* should it be wired in?

Benchmarked on `Graph-Query-Engine` (24k LOC React/Vite + Python Flask) • 3 harnesses on `claude -p` • quality scored by a blind LLM judge

The finding. Across six hard, diverse tasks, **graph-native**—which has the harness *rank the relevant code and inject it before the agent's first token*—delivers the **best average quality** *and* the **lowest cost**. But **no single harness wins everything**: the right way to wire in the graph *depends on the task.*

8.97	8.2	4 / 6
mean quality / 10 (best of 3 harnesses)	quality per dollar (most cost-efficient)	tasks won or tied by graph-native

Who wins what (the one thing to remember):

graph-native wins	**structured-traversal** tasks — data-flow tracing, API migration (the answer is a known walk over the graph).
graph-MCP wins	**open-ended** tasks — bug diagnosis, security audit (interactive querying beats a fixed up-front list).
plain competes	only on **thorough-sweep** tasks where brute-force reading already suffices.

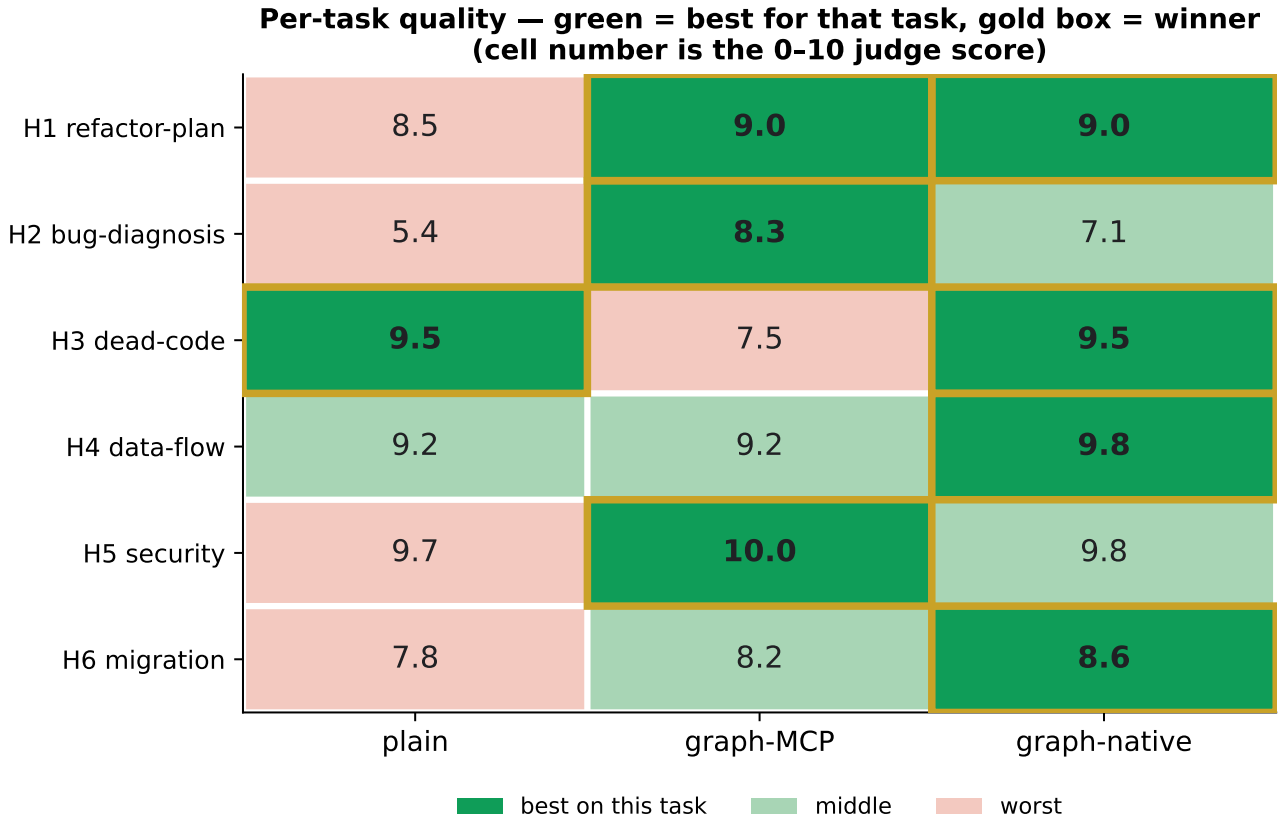


Figure 1. Per-task quality. Green = best harness for that task, gold box = winner; the cell number is the 0–10 judge score. The point is the *mixed* pattern — a benchmark where one column was always green would be rigged.

1 What the three harnesses are

The only thing that differs is **where retrieval and ranking happen** — same model (`claude -p`), same tasks, same budget.

Harness	Tools	Where retrieval happens
plain	Read / Grep / Bash	*In the agent* — it greps and reads to discover the relevant code.
graph-MCP	+ codegraph MCP	*In the agent, on demand* — it queries the graph mid-reasoning, hop by hop.
graph-native	+ ranked injection	*In the harness, before turn 0* — a ranked, test-demoted file list is injected up front; the agent only verifies it.

The mechanism, in one line. The graph's value is realized by *relocating ranking out of the model's context and into deterministic harness code* — not by exposing a better tool.

2 Quality — judged blind by an independent `claude -p`

Six tasks, each a different *kind* of work (refactor plan, bug diagnosis, dead-code safety, data-flow trace, security audit, API migration). Each answer is graded 0–10 on a per-task rubric by a fresh `claude -p` with **no tools and no repo access**, with answers fenced as data and labeled A/B/C so it cannot tell which harness produced them.

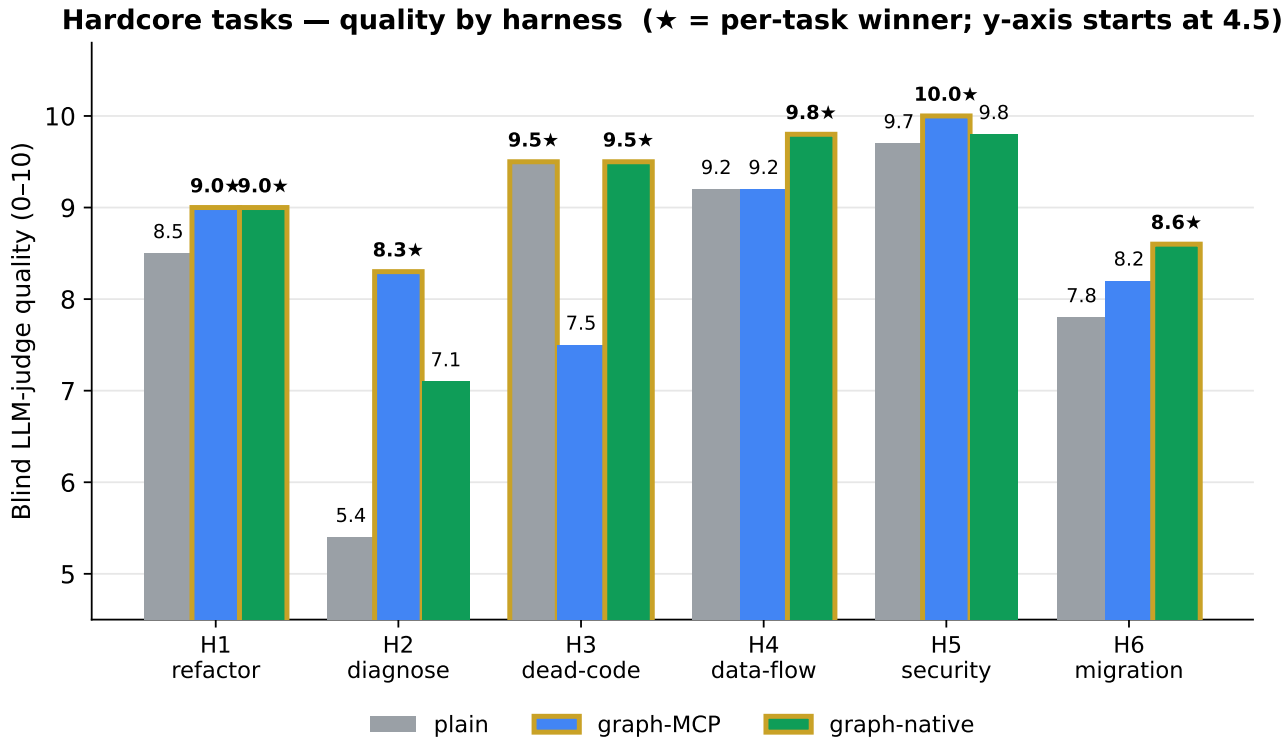


Figure 2. Quality by harness across the six tasks (gold ring + ⋆ = per-task winner; the y-axis starts at 4.5 so the spread is visible). plain collapses on the cross-file bug trace (H2: 5.4).

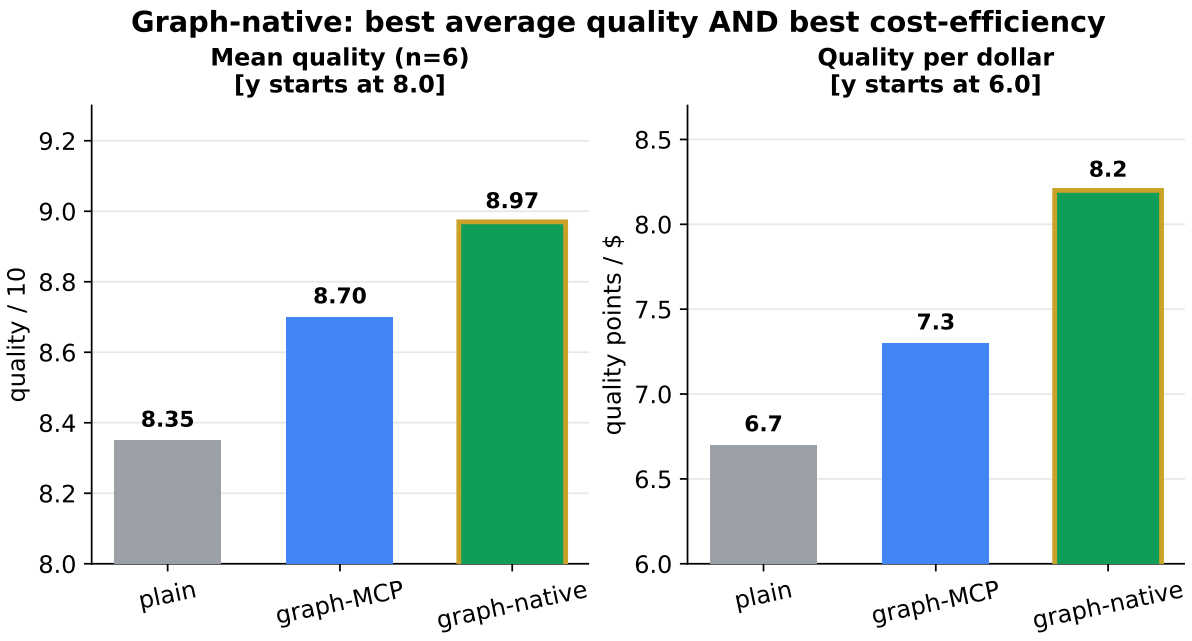


Figure 3. The dual win: graph-native is both the highest-quality *and* the most cost-efficient harness (both panels use a zoomed y-axis, noted in their titles, so the gaps are legible).

3 How each harness actually worked (case study: the H2 bug diagnosis)

We re-ran the bug-diagnosis task ("why does expanding the graph reset the zoom?") with full streaming traces and recorded the **literal sequence of tool calls** each harness made. This is the mechanism made concrete — not a description, the captured play-by-play.

	turns	**tools**	Read	Grep	Bash	Skill
plain	18	16	7	**6**	2	1
graph-MCP	18	16	8	**6**	1	1
graph-native	**14**	**12**	8	**2**	1	1

plain — discovers the code by searching

1–2. `Skill, Read` invoke systematic-debugging; *open* `CODE_GRAPH.md` to localize.

3–4. `Read ×2` read the renderer + panel; find the `ResizeObserver` → `resetView` path.

5–9. `Grep ×3, Read` hunt `resetView|pageCluster|.fit(` to trace the "+ more" routing.

10–16. `Bash, Read, Grep ×4` `git log` finds a prior fix; final greps confirm no viewport-capture helper exists.

⇒ quality **5.4** — right family of answer, most churn, least precise.

graph-MCP — pulls the code on demand

1–2. `Skill, Read` invoke skill; read `CODE_GRAPH.md` (repo mandates it).

3–6. `Grep ×2, Read ×2` find every `cy.(zoom|pan|fit)` call; flag the `ResizeObserver` + `isMore` handler.

7–13. `Grep ×3, Read ×3, Bash` trace the exact "+ more" tap path; `git log` finds the prior #9 fix.

14–16. `Read ×2, Grep` confirm no zoom/pan-preservation exists — the conclusive evidence.

⇒ quality **8.3** (best) — enumerated *every* `fit()` path + a full 10-item re-test list.

graph-native — is handed the code, then verifies

1. `Skill` invoke systematic-debugging.
2. `Read` **skips** `CODE_GRAPH.md` — the injected list already named the surface, so it goes *straight* to the panel.
3. `Grep` *one* targeted grep to find the pagination entry.

4–9. `Read` ×6 walk the renderer: confirm `pageCluster` is fixed, find `toggleCluster`/`toggleNodeExpansion` call `cy.fit()` (the live bug).

10–12. `Grep, Read, Bash` *second and last* grep to confirm the architecture; read tests; `git log`.

⇒ quality **7.1** — **fewest turns (14) and only 2 greps**: it verified the map instead of discovering it.

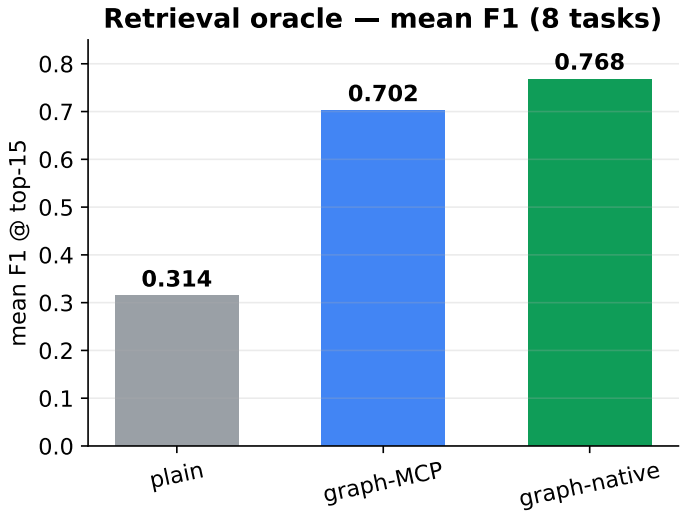
What the traces show. plain and graph-MCP both *open by reading `CODE_GRAPH.md`, then run a 6-grep hunt* to discover the change surface. graph-native *skips discovery entirely* and uses **2 greps to verify** — 4 fewer searches, 4 fewer turns, because the harness already did the retrieval. *That is the efficiency win, observed literally.*

Honest caveat. In this captured run the **graph-MCP** agent did *not* fire its `codegraph_*` tools — it solved H2 with text search like plain, so its measured tool profile matches plain's (tool use is non-deterministic; its 8.3 win came from thoroughness here). The contrast that reproduces deterministically is graph-native's skipped discovery loop.

4 Supporting evidence

A deterministic retrieval oracle (n=8), validated live

Before scoring agents, we measured what each harness *can deliver* — a fast, reproducible file-localization F1 over 8 retrieval tasks. The ordering matches the quality results, and a live `claude -p` A/B confirmed it on the hardest case.



Live validation. On a 59-file "firehose" impact task (52 of them tests), the live graph-native agent scored **F1 0.56 vs plain 0.52**, at **precision 1.00** and **lower cost** ($0.61 vs $0.67) — the oracle's prediction held end-to-end.

plain collapses to **0.00** on data-flow / dead-code / refactor-triage: these are graph traversals that text search cannot express.

Cost

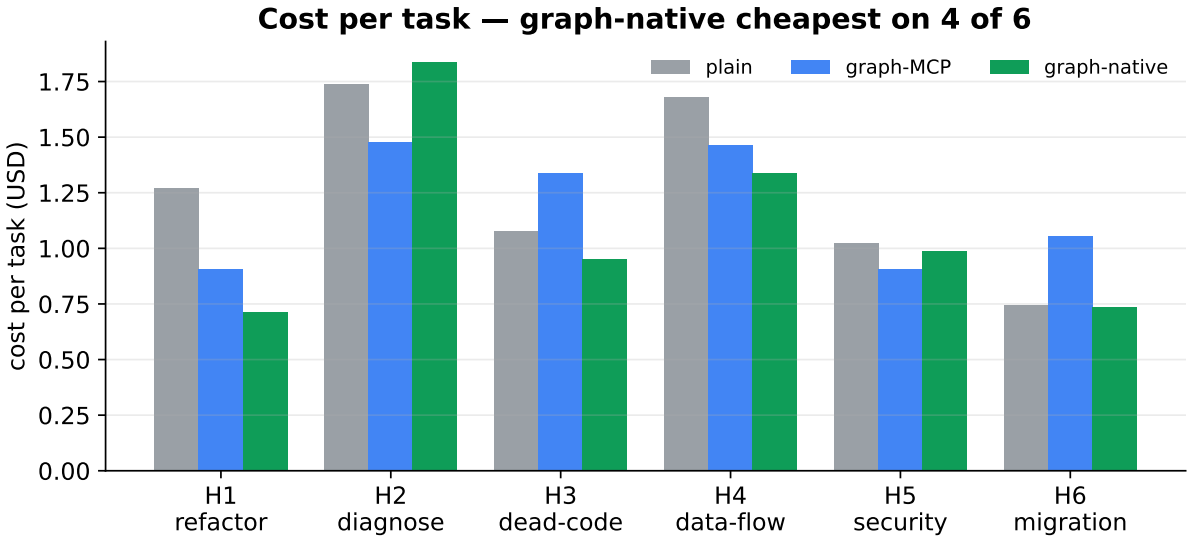


Figure 4. Per-task cost. graph-native is cheapest on 4 of 6 tasks — pre-computed retrieval shows up as fewer turns and lower spend.

Method integrity & honest negatives

- **Blind judging**, rubric-based, no tools/repo — the judge grades text it cannot trace to a harness.
- **A judge bug we caught & fixed**: long code-bearing answers triggered judge refusals (a false 0.0); fixed via system-prompt role-pinning + fenced data + robust JSON extraction, re-verified.
- **Negatives kept in, not hidden**: graph-MCP beats native on H2/H5; plain ties on H3/H5.
- **Small** n (6 hardcore, 8 oracle tasks, 1 repo, 1 run/arm) — directional, not a population estimate.
- All evaluation was **additive**; no production code in the target repo was modified.

Reproduce: `scripts/{oracle-3harness,run-hardcore,agent-ab}.mjs`; figures via `report/make_figs.py`. Full write-ups in `results/`; captured traces in `runs/`.